

ESDC Lab Project: Tron 2D

Project specifications

We are going to develop the game *Tron 2d*. First of all, we are going to explain roughly what this game is about. *Tron 2d* is a game based on the famous movie franchise *Tron*. It is a game for two players where one has to defeat the other. We want that each player plays in its own computer. Given a limited area, e.g. a rectangle, both opponents appear in an initial position inside the limiting area. Every periodic amount of time, the position of both players is updated. The players cannot remain in the same position, they always move forward or turn in a perpendicular direction (they cannot move diagonally). If no turn is indicated by the player, it automatically moves forward. The most important rule is the following: once a player have passed through a certain point for first time, he cannot pass again. At the same time, it is not allowed that he passes through a certain point where the opponent has already passed. In order to make the game enjoyable and dynamic, and not a tedious exercise of memory, the limiting area and the players are showed in a screen, and the trajectory of each player is colored, in order to let the players know where they should not move to. Knowing this, the first player who get to one of these already “passed” points or get out of the limiting area will lose. It could happen that both players moved to a forbidden point at the same time, then we would have a tie.

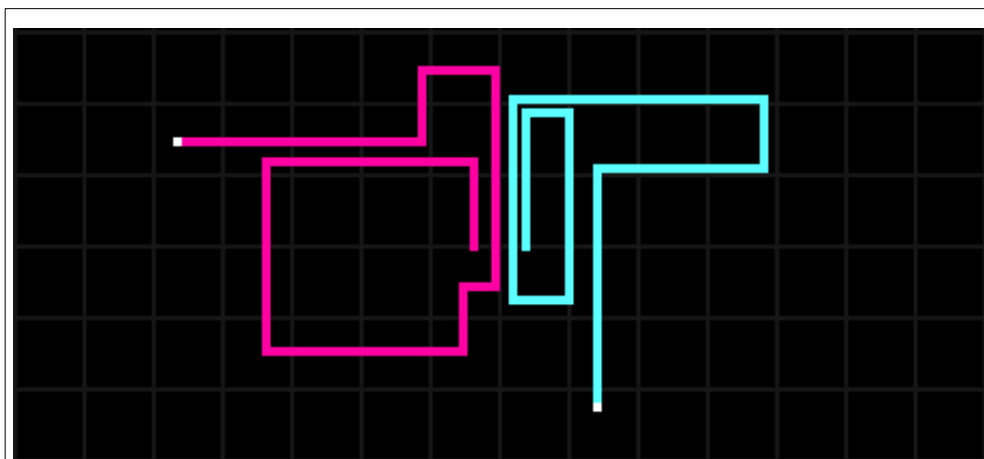


Figure 1: Screenshot of the game, by <https://noobtuts.com/>

Note that, in order to be a playable game, the screen of each player must be really synchronized with the other, so that each player has reliable information of the already passed points by the opponent. Every period of time, the new position of the player is communicated to the opponent using a RS-232 interface. Each player should somehow compute if its selected movement collide with any of the colored trajectories (or the contour of the limiting area).

Now we are going to explain how to implement it. First of all, we specify the timing of the game. Both players, in order to play and interact, will use an extension keyboard.



Figure 2: Keyboard that will be used connected to the board.

In order to start the game, each player should press the asterisk symbol. Establishing the beginning of the game is a critical step, as long as good synchronization between screens is required. We will do the following: once a player has sent the asterisk, a timeout has passed and the opponent asterisk has been received, three seconds pass and the game starts. In the next cronogram we illustrate this process:

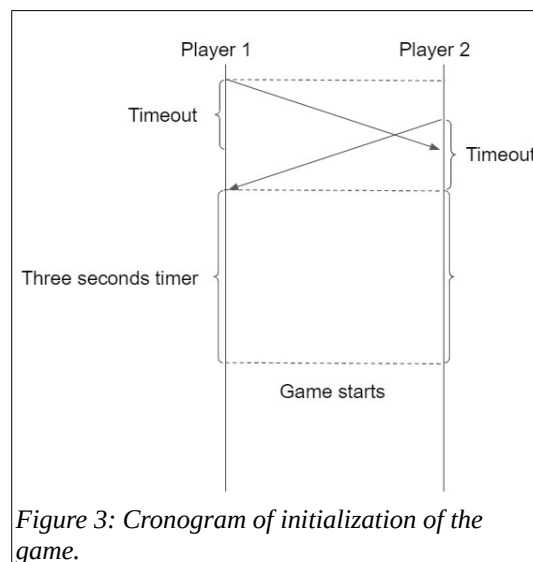


Figure 3: Cronogram of initialization of the game.

Note that the timeout corresponds to the time of sending the asterisks, and, taking into account the clock frequency and the envisioned length of the sent messages, it will be 38 ms. Also note that RS-232 will work in bidirectional mode, so no collisions are expected. If there is an error when sending the asterisk, the system will be stagnated and it will be necessary to reset in both sides.

Once this initialization is done, the game starts. We will use a VGA monitor, with a resolution of 640x480. However, we will reduce the resolution by sixteen, defining an artificial pixel formed by 16x16 real pixels, so we would have an operative resolution of 40x30. The initial points of the players will be (14,15) and (26,15), beginning the numbering of pixels at 0. The head of a player will be one artificial pixel, and each movement will imply moving to an adjacent artificial pixel. Each player should know which player is, either 1 or 2, a priori. That is, before the starting the asterisks initialization, the player should already know which its initial position will be. This is the only asymmetry between the players: the identifier. Each player will know its ID (1 or 2) according to the position of a certain switch in the Altera board. If both players would select the same ID, the game would become unplayable, as long as both players do not have opponent's position.

When the game is started and the initial position displayed, the same time sequence will repeat until the end of the game. We will have a period of time of refresh, that is, the period that the game will update the positions. This period will have a maximum time the players are allowed to transmit, in order to avoid desynchronizations. This period of refresh will be 250 ms, i.e., 4 refreshes per second.

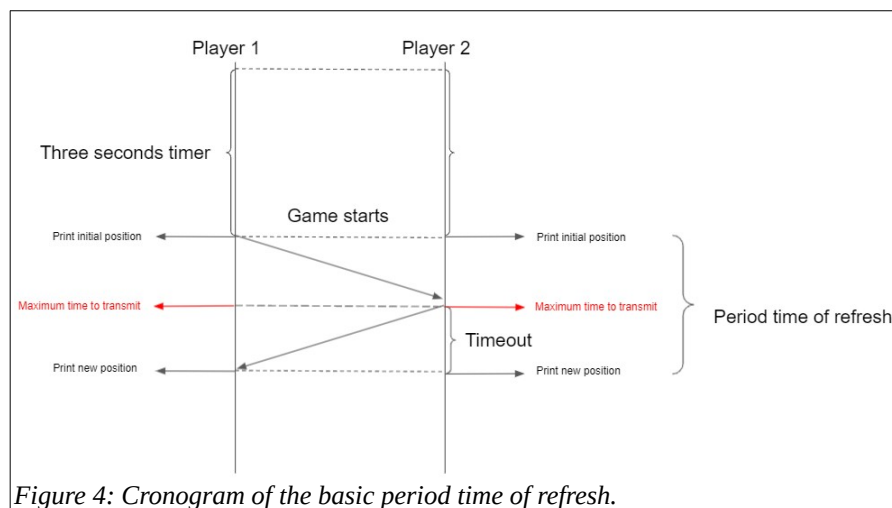


Figure 4: Cronogram of the basic period time of refresh.

Now we are going to define the frame used to convey the information. We will use a really compact frame of 3 bits. One of the main ideas to get such a reduced size is to send the Δx or Δy , as long as we know the initial position of the opponent. In the following table we specify the frames and their meaning:

Bits	Meaning
100	Asterisc
000	$\Delta x = 1 \Delta y = 0$
001	$\Delta x = -1 \Delta y = 0$
010	$\Delta y = 1 \Delta x = 0$
011	$\Delta y = -1 \Delta x = 0$

Taking into account the available keys, we have the following relation: 000 $\equiv$ '6', 001 $\equiv$ '4', 010 $\equiv$ '2' and 011 $\equiv$ '8'. Two final notes about the protocol. First off, if no valid key is pressed, the player will automatically send a forward movement in the maximum time to transmit.

But, what is an invalid key? On the one the hand, all the unused key of the keyboard (1, 3, 5, etc.). On the other hand, a player won't be able to go back without before turning left or right. That is, if a player has moved $\Delta x = 1 \Delta y = 0$, cannot move $\Delta x = -1 \Delta y = 0$ until $\Delta y = 1 \Delta x = 0$ or $\Delta y = -1 \Delta x = 0$ has been pressed. Finally, if no key is pressed at the initial position, player one will move $\Delta x = 1 \Delta y = 0$ and player two $\Delta x = -1 \Delta y = 0$ by default. If there is an error in the sent move, leading to an unused framed (for instance, 111), the system will discard it and will consider the opponent movement is to continue with the same Δx and Δy . This situation could produce a chaotic behavior in the game, as long as the registered occupied cells could be different in each player.

Each player will save the trajectory and new movement from himself and the opponent, and will assess the output of the new movement: he wins, the opponent wins, tie or the game continues.

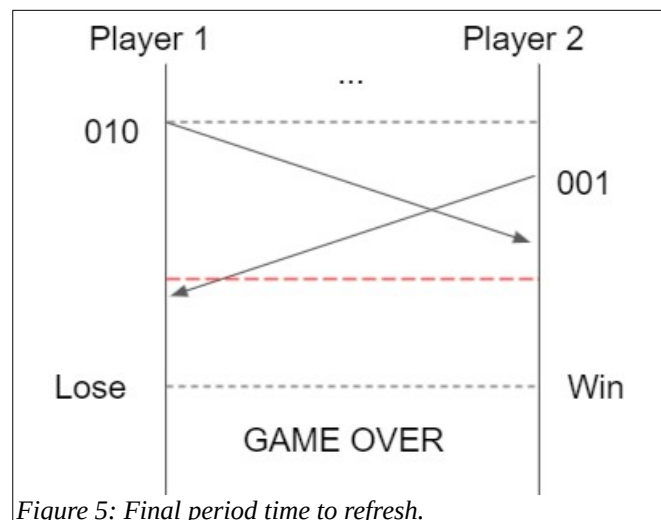
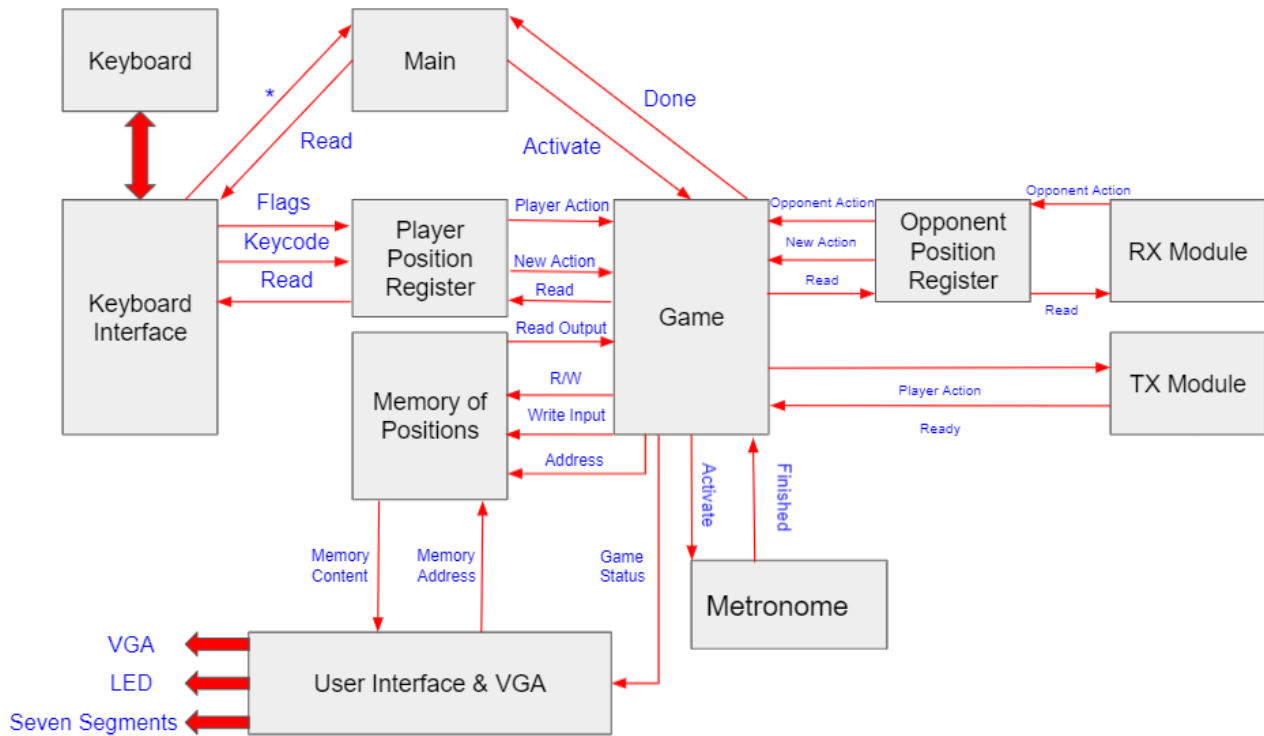


Figure 5: Final period time to refresh.

If they want to play again, repeat the process pressing the asterisk. The VGA screen will erase the trajectories of the previous game.

In order to indicate if the player has won, lost, or it is a tie, the LEDs of the board will be used. The resulting LED will be turned on until the player pulses the asterisk.

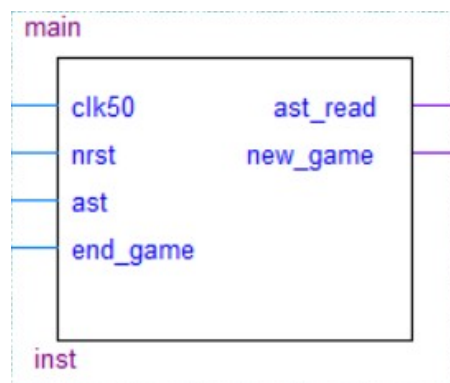
Version 1 block diagram



The keyboard interface sends an asterisk to the Main, indicating we want to start a new game. The Main acknowledges the reception of the asterisk by a Read signal. Then, Main activates Game and all the mechanisms of the game start. On the one hand, Game will interact with the Player Position Register (PSR). PSR will indicate to Game the a new action has been uploaded, and Game will acknowledge when the movement is taken into account (it could be that a new movement is in the register in the prohibited time of transmission, look at Project Specs). On the other hand, Game will deal similarly with the Opponent Position Register. A fundamental part of the design is the Memory of Postions. The memory has as many cells as positions the game has, i.e., $40 \times 30 = 1200$, and Game will need to be able to read, in order to check if a certain position is already occupied, and write, in order to update the trajectories of the players. At the same time, this memory will be used in order to refresh the VGA screen. In order to allow a simultaneous read and write by the Game module and read by the User Interface Module, a three port memory will be used. The User Interface Module, besides the Memory of Positions, will interact with the Game module in order to indicate to the user the result of the game through LEDs (win, lose or tie) and complementary things, such as visualizing the initial three seconds timer of the beginning of the game through seven segments. We also have the communications modules. TX Module will interact with the Game Module, whereas the RX Module will interact with the Opponent Position Register, using similar input and output signals. Finally, we have the Metronome module, that indicates to the Game module the timings of timeout and refresh time minus timeout.

Version 2 block diagram

Main



ast: it indicates whether asterisk has been pressed by the user.

end_game: it indicates whether a certain game is over.

ast_read: signal active high, synchronous. Used as ACK signal for the KEYBOARD block, to indicate that the * has been read.

new_game: signal active high, synchronous. It indicates to Game when to start a new game.

Player Register



key_code[3..0]: code of the key pressed by the user. Signal connected to the keyboard block.

read_move: it indicates when the Game module has read the x and y outputs.

new_code: it indicates when the Keyboard Interface has a new pressed key to transmit.

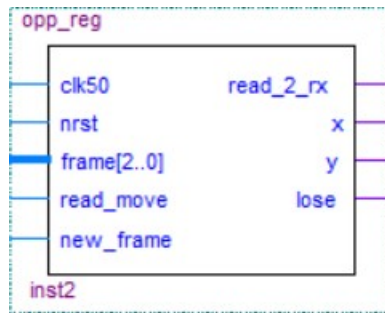
key_read: ACK awakening signal for the keyboard when a key has been read by this block.

new_move: a flag signal that indicates to the Game module that a new movement has been played.

x: indicates the variation of x of the new movement to the Game Module.

y: indicates the variation of y of the new movement to the Game Module.

Opponent Register



frame[2..0]: 3 bits that indicate the new movement or state of the opponent.

read_move: it indicates when the Game module has read the x and y outputs.

new_frame: a flag signal from the RX module that indicates that a new frame is received.

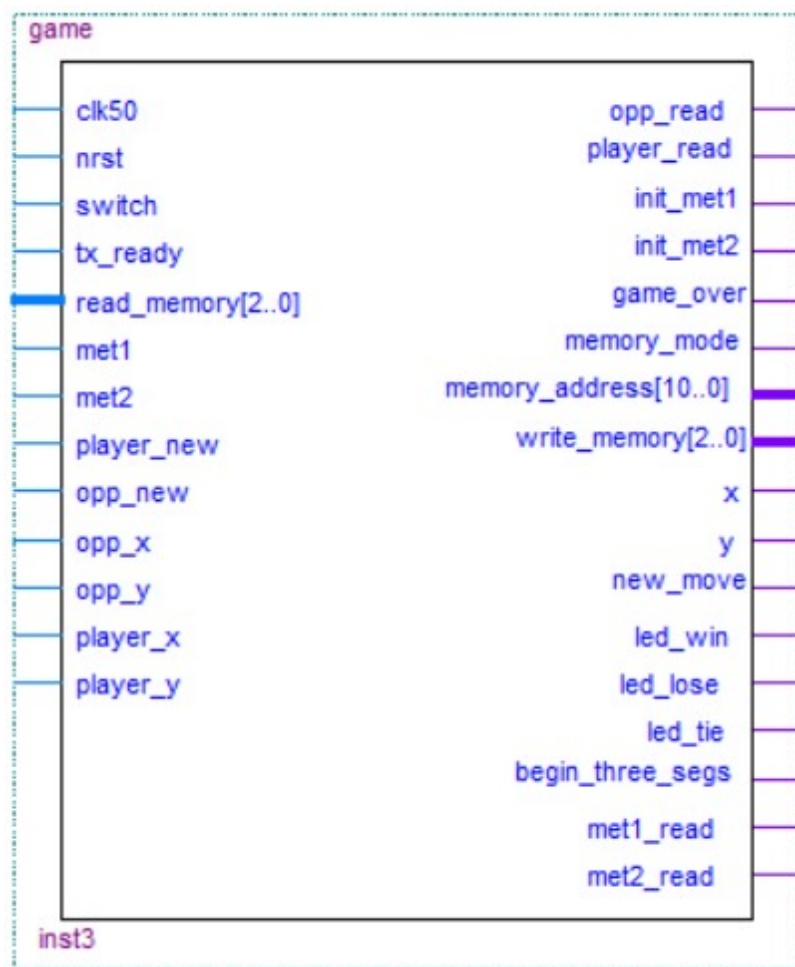
read_2_rx: it indicates to the RX module that the message has been read.

x: it indicates the variation of x of the opponent's new movement to the Game Module.

y: it indicates the variation of y of the opponent's new movement to the Game Module.

Lose: it indicates to the Game Module whether the opponent has sent a lose.

Game



switch: it indicates the ID of the player, 1 if it is high, 2 if it is low.

tx_ready: input from the TX module that indicates that a frame has been properly sent.

read_memory[2..0]: lecture from a cell of the Memory of positions.

Met1: flag signal that indicates that timing referring refresh time minus timeout is over.

Met2: flag signal that indicates that timing referring timeout is over.

player_new: flag signal that indicates a new movement by the player.

opp_new: flag signal that indicates a new movement by the opponent.

opp_x: it indicates the x variation of the last movement of the opponent.

opp_y: it indicates the y variation of the last movement of the opponent.

player_x: it indicates the x variation of the last movement of the player.

player_y: it indicates the y variation of the last movement of the player.

opp_read: it indicates to Opponent Position Register that the movement has been properly caught by Game Module.

player_read: it indicates to Player Position Register that the movement has been properly caught by Game Module.

Init_met1: awakening signal that indicates to the metronome module to start timing referring to refresh time minus timeout.

Init_met2: awakening signal that indicates to the metronome module to start timing referring to timeout.

game_over: flag that indicates the end of the game to the Main module.

memory_mode: it indicates to memory the kind of access, write (high) or read (low).

memory_address[10..0]: it indicates to memory the position to access.

Write_memory[2..0]: the bits to write in a cell of memory.

X: variation of x of the new movement of the player to be sent.

Y: variation of y of the new movement of the player to be sent.

new_move: it indicates to the transmission module whether there is a new movement to send.

led_x: bits that indicates the result of the game.

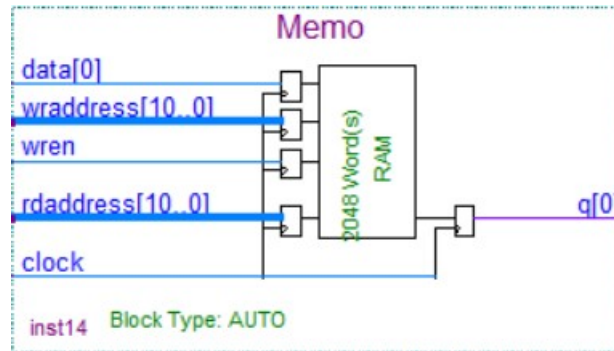
begin_three_secs: it indicates to the User Interface module when to start the initial three seconds counter.

met1_read: it indicates the met1 input has been read.

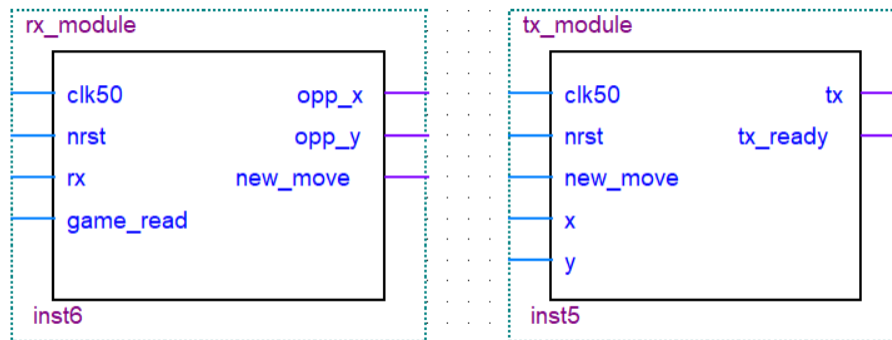
met2_read: it indicates the met2 input has been read.

Memory Positions

This block will consist of the megafunction of MegaWizard so-called 2-port RAM:



Communications Modules



rx_module

rx: serial input connected to the opponent's transmitter.

game_read: awakening signal that indicates the game has read the new movement.

opp_x, opp_y: flag signals that indicate the movement variation of the opponent.

new_move: flag signal that indicates to the Game that a new movement is received.

tx_module

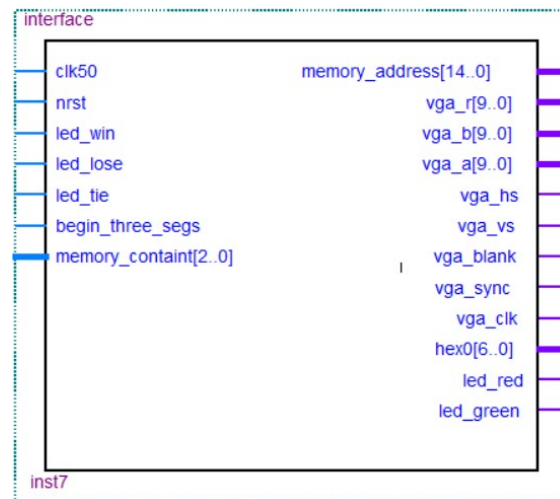
new_move: awakening signal from Game that indicates that a new frame needs to be sent.

x, y: flag signals from Game that indicate what frame the player wants to send.

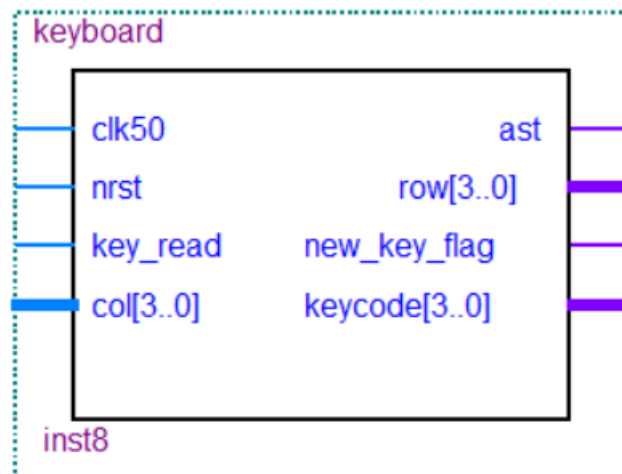
tx: serial output to the opponent's receiver.

tx_ready: signal active high. This signal is active when the transmitter is not sending any frame, indicating that a new frame can be sent.

Interface



Keyboard Interface



COL[3..0], ROW[3..0]: Signals to connect to the external keyboard

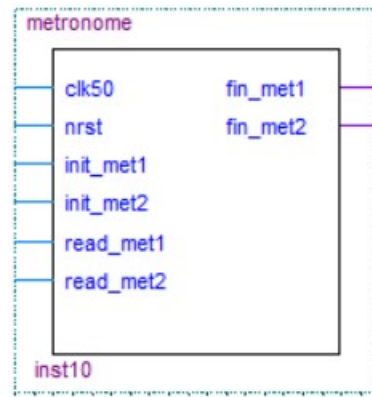
Key_read: The system that reads the key, acknowledges that it has been properly read by activating this input. Synchronous signal, active high. It must be active just one clock period and, when active, deactivates all the output flag signals.

new_key_flag: any key is pressed.

key_code: code of the key pressed in hexadecimal. When the user presses any key from 0 to 9, keycode contains the corresponding hexadecimal code. * has de code E and # has de code F.

ast: * is pressed.

Metronome



init_met1: an awakening signal that indicates to begin timing referring to refresh time minus timeout.

init_met2: an awakening signal that indicates to begin timing referring to timeout.

read_met1: an awakening signal that indicates the Game has read the flag output fin_met1.

read_met2: an awakening signal that indicates the Game has read the flag output fin_met2.

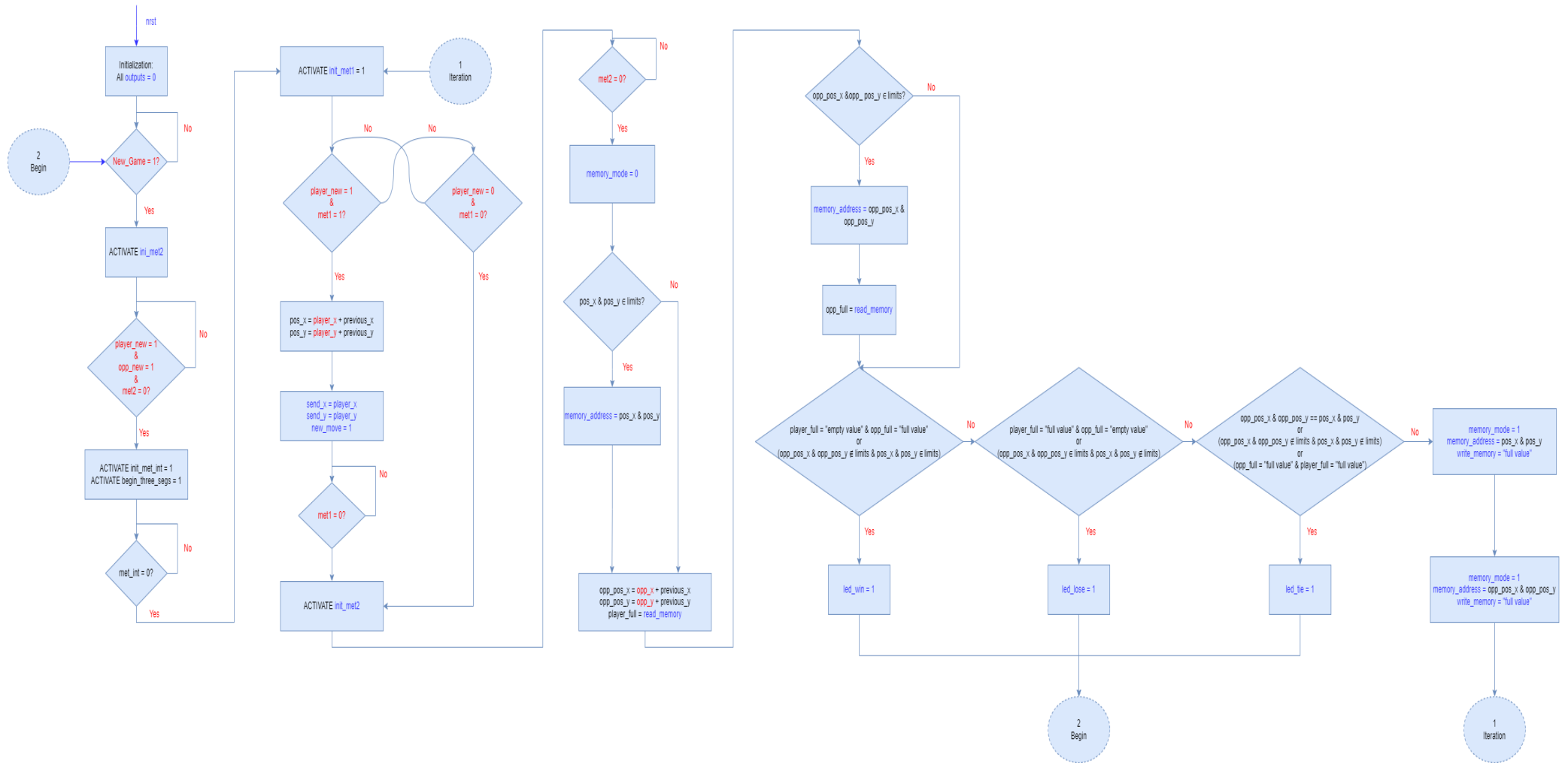
fin_met1: flag signal that indicates the end of met1 timing.

fin_met2: flag signal that indicates the end of met 2 timing.

Algorithms

Block Game

Red: inputs. Blue: outputs. Black: internal signals.



Work Plan

Delivery 1: simulation of blocks Player Position Register, Opponent Position Register, Rx Module, Tx Module and Metronome.

Delivery 2: simulation of blocks Game, Memory of Positions and the VGA interface. Partial experimental validation of Game, Memory of Positions and the VGA Interface altogether.

Link to Design Files

<https://drive.google.com/drive/folders/1a96653XuBgcIMsF4KycRhJ2pan9eAIny?usp=sharing>